

Final Project Report

UCS310 – Database Management System

FarmFresh

Submitted by

1024170097 Jaskaran Singh Bedi

1024170120 Vansh Wadhwa

1024170013 Abhijay Nagal



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Computer Science and Engineering Department

TIET, Patiala

Jan-June 2026

INDEX

Sr no.	Title
1	Introduction
2	System Overview
3	ER Diagram
4	Relational Schema
5	Database Design and DBMS Concepts
6	Normalization
7	Triggers and Cursors
8	Future Scope
9	Conclusion
10	Technology Stack

1. Introduction

FarmFresh is a DBMS-based e-commerce platform developed to connect farmers directly with customers for buying and selling dairy products such as milk, curd, paneer, butter, ghee, and sweets. The system helps farmers manage products and stock efficiently while allowing customers to browse items, add products to the cart, and place orders online.

The project uses PostgreSQL (Supabase) as the database backend and implements relational database concepts using tables such as users, products, categories, orders, cart_items, and order_items. These tables are connected using primary keys and foreign keys to maintain data integrity and proper relationships between entities.

FarmFresh also demonstrates advanced DBMS concepts such as triggers, cursors, and stored procedures. Triggers are used for tasks like updating timestamps, validating order status, restoring stock after cancellations, and logging low-stock products. Cursor-based procedures are used for processing product and order data efficiently.

The project provides a practical implementation of database management concepts while creating a simple, efficient, and reliable online dairy marketplace system.



2. System Overview

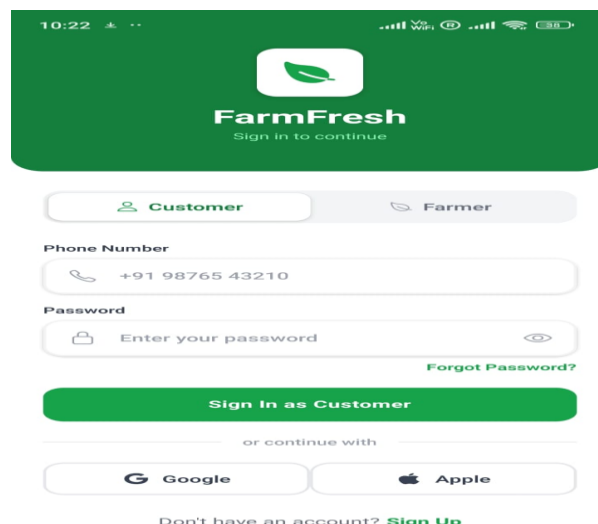
FarmFresh is a full-stack DBMS-based dairy marketplace system that connects farmers directly with customers through an online platform. The system is developed using Node.js, Express.js, and PostgreSQL (Supabase) and follows a REST API architecture for communication between the frontend and backend. The platform supports two types of users: customers and farmers. Customers can browse dairy products, manage carts, place orders, and save delivery addresses, while farmers can manage products, monitor stock, and track sales performance.

The database is designed using relational tables such as users, products, categories, cart_items, orders, and order_items. Primary keys and foreign keys are used to maintain relationships and ensure data consistency. The system also implements advanced DBMS features like triggers, cursors, and stored procedures for stock management, order tracking, automatic timestamp updates, and low-stock monitoring.

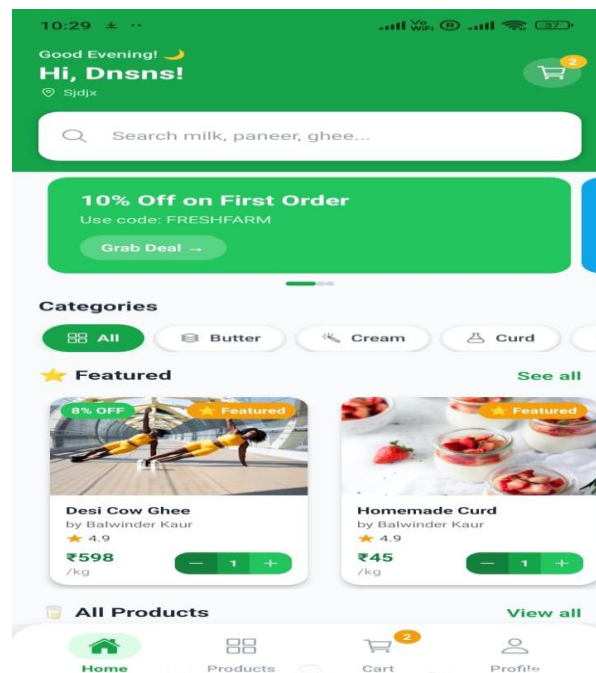
The application includes secure authentication using JWT (JSON Web Token) and encrypted password storage using bcrypt hashing. Role-based access control is implemented to provide separate functionalities for customers and farmers.

Core Features

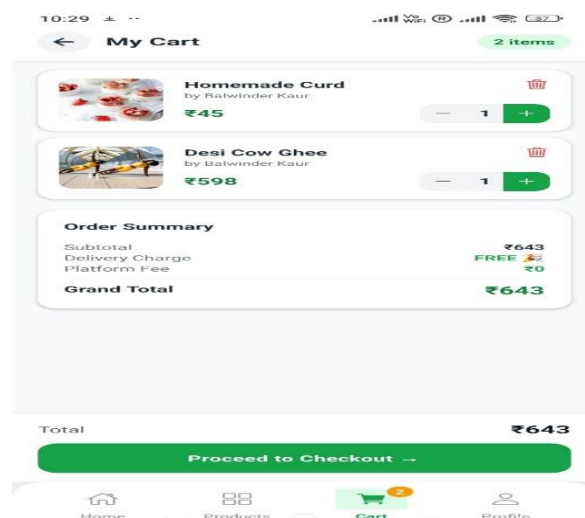
- User Authentication and Authorization:** The system provides secure user registration and login for both customers and farmers using JWT authentication and encrypted passwords. It also supports profile management and address handling.



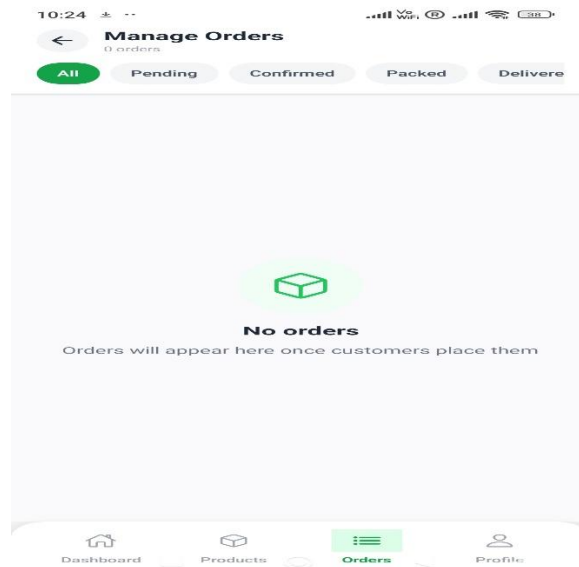
2. **Product Management:** Farmers can add, update, delete, and manage dairy products with details such as category, price, stock, discount, and product images. Customers can search and filter products based on categories and keywords.



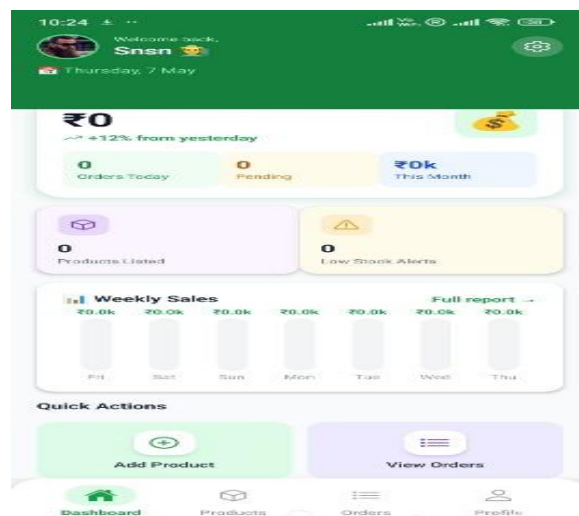
3. **Shopping Cart System:** Customers can add products to the cart, update quantities, remove items, and calculate total costs dynamically. The system checks stock availability before adding products to the cart.



4. **Order Management:** Customers can place orders and farmers can update order statuses such as confirmed, packed, delivered, or cancelled.



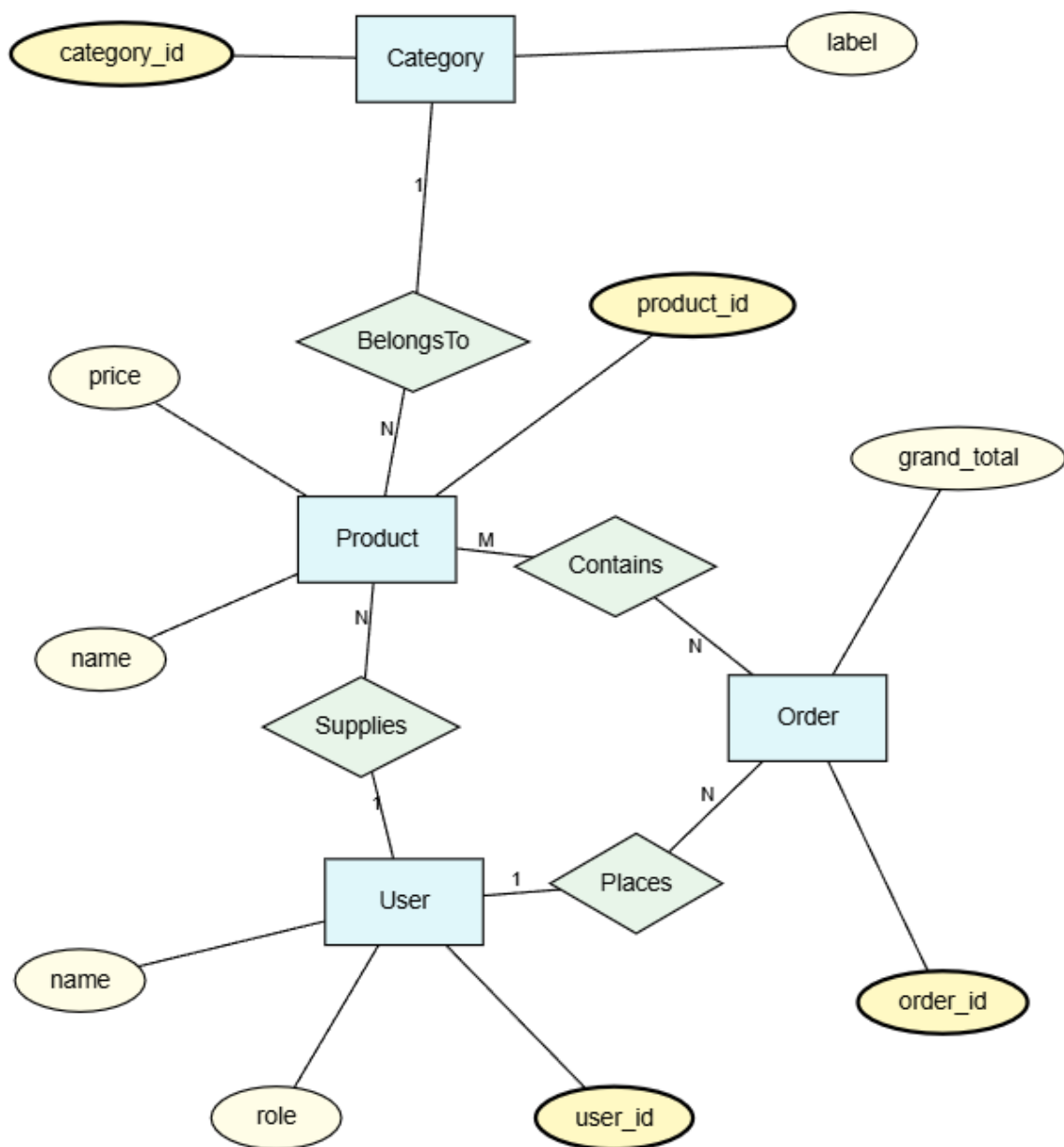
5. **Farmer Dashboard and Analytics:** Farmers can view sales statistics, weekly sales reports, monthly earnings, pending orders, and low-stock product alerts through dashboard APIs.



6. **Database Automation using Triggers and Cursors:** The project uses triggers and cursor-based stored procedures for automated tasks such as updating timestamps, validating order status transitions, recalculating farmer ratings, and logging low-stock products.

3. ER Diagram

The Entity-Relationship (ER) model of the FarmFresh Dairy Marketplace represents the core data structure of the system by defining entities, their attributes, and the relationships between them. The design ensures minimal redundancy, clear role separation, and efficient handling of transactions.



4. Relational Schema

The relational schema of the FarmFresh Dairy Marketplace is derived from the ER model and defines the structure of tables along with primary keys and foreign key relationships. The schema is designed to ensure data consistency, eliminate redundancy, and support efficient querying.

Tables and Schema

```
CREATE TABLE categories (category_id VARCHAR(20) PRIMARY KEY, label VARCHAR(50) NOT NULL, icon VARCHAR(100));
```

```
CREATE TABLE users (  
  user_id SERIAL PRIMARY KEY, name VARCHAR(100) NOT NULL, email VARCHAR(150) UNIQUE, phone VARCHAR(20) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL, role VARCHAR(10) NOT NULL, avatar_url VARCHAR(500), address VARCHAR(500),  
  location VARCHAR(200), about TEXT, years_of_experience INT, verified INT DEFAULT 0, bank_account VARCHAR(100),  
  upi_id VARCHAR(100), rating NUMERIC(3,1) DEFAULT 0, review_count INT DEFAULT 0, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, is_active INT DEFAULT 1  
);
```

```
CREATE TABLE products (  
  product_id SERIAL PRIMARY KEY, farmer_id INT REFERENCES users(user_id), category_id VARCHAR(20) REFERENCES categories(category_id),  
  name VARCHAR(200) NOT NULL, description TEXT, price NUMERIC(10,2) NOT NULL, unit VARCHAR(20) NOT NULL, stock INT DEFAULT 0,  
  discount NUMERIC(5,2) DEFAULT 0, image_url VARCHAR(500), featured INT DEFAULT 0, rating NUMERIC(3,1) DEFAULT 0,  
  review_count INT DEFAULT 0, tags VARCHAR(500), is_active INT DEFAULT 1, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE cart_items (  
  cart_item_id SERIAL PRIMARY KEY, user_id INT REFERENCES users(user_id), product_id INT REFERENCES products(product_id),  
  quantity INT NOT NULL, added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, UNIQUE(user_id, product_id)  
);
```

```
CREATE TABLE orders (  
  order_id SERIAL PRIMARY KEY, order_ref VARCHAR(20) UNIQUE NOT NULL, customer_id INT REFERENCES users(user_id),  
  farmer_id INT REFERENCES users(user_id), delivery_address VARCHAR(500) NOT NULL, payment_mode VARCHAR(20) NOT NULL,  
  special_note VARCHAR(500), subtotal NUMERIC(10,2) NOT NULL, delivery_charge NUMERIC(10,2) DEFAULT 0,  
  grand_total NUMERIC(10,2) NOT NULL, status VARCHAR(30) DEFAULT 'confirmed', created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE order_items (  
  order_item_id SERIAL PRIMARY KEY, order_id INT REFERENCES orders(order_id), product_id INT REFERENCES products(product_id),  
  product_name VARCHAR(200) NOT NULL, quantity INT NOT NULL, unit_price NUMERIC(10,2) NOT NULL, unit VARCHAR(20) NOT NULL  
);
```

```
CREATE TABLE user_addresses (  
  address_id SERIAL PRIMARY KEY,  
  user_id INTEGER REFERENCES users(user_id) ON DELETE CASCADE,  
  label VARCHAR(50) DEFAULT 'Home', -- e.g. Home, Office, Other  
  address TEXT NOT NULL,  
  is_default INTEGER DEFAULT 0,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```


Schema Characteristics

The FarmFresh database schema is designed using the relational database model to ensure efficient storage, retrieval, and management of data. The schema consists of multiple interconnected tables such as users, products, categories, cart_items, orders, order_items, and user_addresses. Relationships between tables are maintained using primary keys and foreign keys to preserve referential integrity and consistency.

The schema follows normalization principles to minimize data redundancy and improve database efficiency. Most tables are structured up to Third Normal Form (3NF), where repeating groups are removed, partial dependencies are eliminated, and non-key attributes depend only on the primary key.

The database supports two major user roles: customers and farmers. The users table stores common user information, while products and orders establish relationships between farmers and customers. The order_items table resolves the many-to-many relationship between orders and products.

Important schema characteristics include:

- **Primary Keys:** Used to uniquely identify each record in tables such as user_id, product_id, and order_id.
- **Foreign Keys:** Used to establish relationships between tables and maintain referential integrity.
- **Constraints:** NOT NULL, UNIQUE, and DEFAULT constraints are used for data validation and consistency.
- **Transaction Management:** Transactions using BEGIN and COMMIT ensure safe order placement and stock updates.
- **Triggers and Stored Procedures:** Implemented for automatic timestamp updates, stock validation, low-stock logging, and order status management.
- **Audit Tables:** order_status_log and low_stock_log maintain historical records for monitoring and tracking system activities.
- **Scalability:** The schema is modular and can be extended easily with features like payments, reviews, or delivery tracking in the future.

The schema is optimized to provide secure, reliable, and efficient database operations for the FarmFresh dairy marketplace system.

5. Database Design and DBMS Concepts

The FarmFresh project is designed using the relational database model to ensure efficient data storage, consistency, and secure transaction handling. The database contains multiple related tables such as users, products, categories, orders, order_items, and cart_items, which are connected using keys and constraints to maintain referential integrity.

The system uses different types of keys for proper database management. Primary keys such as user_id, product_id, and order_id uniquely identify records in each table. Foreign keys are used to establish relationships between tables, for example, farmer_id in the products table references the users table, while product_id in order_items references the products table. Candidate keys such as email and phone number in the users table are used to uniquely identify users.

The database also applies several constraints to maintain data accuracy and consistency. NOT NULL constraints ensure mandatory fields are not left empty, UNIQUE constraints prevent duplicate entries such as duplicate phone numbers or emails, and DEFAULT constraints assign automatic values to attributes like ratings, timestamps, and stock values. Foreign key constraints help preserve valid relationships among tables.

Functional dependencies are maintained throughout the schema design. For example:

- user_id → name, phone, role
- product_id → name, price, stock
- order_id → order_ref, status, grand_total

These dependencies help achieve normalization and reduce data redundancy.

The project uses various SQL operations such as INSERT, UPDATE, DELETE, and SELECT for managing database records. Advanced SQL concepts like JOIN operations are heavily used to combine related data from multiple tables. For example, product details are joined with farmer information, and orders are joined with order_items for complete order retrieval.

Transaction management is implemented during order placement using BEGIN and COMMIT statements. This ensures that all operations such as stock deduction, order creation, and cart clearing are executed safely as a single

transaction. If any error occurs, the transaction is rolled back to maintain database consistency.

The FarmFresh database system also follows ACID properties:

- **Atomicity:** Ensures complete execution of transactions.
- **Consistency:** Maintains valid database states through constraints and triggers.
- **Isolation:** Prevents conflicts during concurrent transactions.
- **Durability:** Ensures committed transactions are permanently stored.

Overall, the database design and DBMS concepts used in FarmFresh provide reliability, security, efficient data handling, and strong relational integrity for the system.

6. Normalization

The FarmFresh database schema is normalized to the **Third Normal Form (3NF)** to reduce data redundancy, improve data consistency, and ensure efficient database operations. The normalization process was applied while designing tables such as users, products, orders, cart_items, and order_items.

First Normal Form (1NF)

The schema satisfies First Normal Form because:

- Each table contains atomic values only.
- There are no repeating groups or multi-valued attributes.
- Every row has a unique primary key.

Example:

- In the products table, attributes such as price, stock, and discount store single values only.
- Tables like cart_items and order_items store one product per row instead of multiple products in a single field.

Second Normal Form (2NF)

The schema satisfies Second Normal Form because:

- It is already in 1NF.
- All non-key attributes are fully dependent on the entire primary key.
- Partial dependency is removed.

Example:

- In order_items, attributes such as quantity and unit_price depend completely on order_item_id and the associated order-product record.
- Product details are stored separately in the products table instead of duplicating them inside orders.

Third Normal Form (3NF)

The schema satisfies Third Normal Form because:

- It is already in 2NF.

- There are no transitive dependencies.
- Non-key attributes depend only on the primary key and not on other non-key attributes.

Example:

- Customer information is stored in the users table, while order-related information is stored separately in the orders table.
- Category details are separated into the categories table instead of repeatedly storing category information in products.

Why Higher Normalization Was Not Applied

The database was not normalized beyond 3NF because further normalization, such as Boyce-Codd Normal Form (BCNF) or Fourth Normal Form (4NF), would increase query complexity and require excessive table joins. This could reduce performance for frequently used operations such as product browsing, cart management, and order retrieval.

Some controlled redundancy was intentionally maintained for better performance and simpler query execution. For example:

- product_name is stored in order_items even though product details exist in products. This preserves historical order information even if the original product data changes later.
- Farmer ratings and review counts are stored in the users table for faster retrieval instead of calculating them repeatedly.

Therefore, 3NF provides a balance between normalization, performance, maintainability, and practical application requirements for the FarmFresh system.

7. Triggers and Cursors

The FarmFresh project uses triggers, cursors, and stored procedures in PostgreSQL to automate database operations and maintain data consistency. These are implemented using PL/pgSQL functions.

Triggers Used

- **Auto Timestamp Trigger:** Automatically updates the `updated_at` field whenever records are modified.
- **Stock Guard Trigger:** Prevents product stock from becoming negative.
- **Order Cancellation Trigger:** Restores product stock when an order is cancelled.
- **Order Status Log Trigger:** Stores old and new order statuses in the `order_status_log` table.
- **Low Stock Trigger:** Logs products whose stock falls below 5 units.
- **Status Validation Trigger:** Ensures valid order status transitions.

Cursors Used

- **Restore Stock Cursor:** Iterates through cancelled order items and restores stock.
- **Farmer Rating Cursor:** Calculates average farmer ratings from product reviews.
- **Low Stock Scan Cursor:** Scans low-stock products and logs them automatically.

Advantages

- Automates repetitive tasks
- Maintains database consistency
- Improves stock and order management
- Provides audit logging and monitoring
- Supports efficient row-by-row processing

Thus, triggers and cursors help make the FarmFresh database system reliable and efficient.

8. Future Scope

The FarmFresh system can be further enhanced by adding more advanced features and improving scalability. Future developments may include online payment gateway integration, real-time order tracking, customer review and feedback systems, and AI-based product recommendations.

The platform can also be expanded to support multiple product categories beyond dairy products, allowing farmers to sell vegetables, fruits, and other organic products. Features such as multilingual support, push notifications, and live chat between customers and farmers can improve user experience further.

From the database perspective, additional modules such as automated inventory prediction and data visualization dashboards can be implemented.

In the future, the system may integrate machine learning techniques for demand forecasting and smart stock management, making the platform more efficient, scalable, and intelligent.

9. Conclusion

The FarmFresh project successfully demonstrates the implementation of a DBMS-based dairy marketplace system that connects farmers directly with customers through an online platform. The system provides functionalities such as user authentication, product management, cart handling, order processing, and farmer analytics in an efficient and user-friendly manner.

The project effectively applies important Database Management System concepts including relational schema design, normalization, primary and foreign keys, transactions, triggers, cursors, and stored procedures. Security features such as JWT authentication and password hashing further improve system reliability and data protection.

The database design ensures data consistency, minimizes redundancy, and supports efficient operations for managing products, customers, and orders. Overall, FarmFresh provides a practical and scalable solution for digital dairy product management while demonstrating the real-world application of DBMS concepts and backend technologies.

10. Technology Stack

The FarmFresh project is developed using a full-stack technology architecture for efficient frontend, backend, and database management.

Frontend

- React Native was used for developing the mobile application interface.
- It provides a responsive and user-friendly experience for customers and farmers.

Backend

- Node.js was used as the server-side runtime environment.
- Express.js was used to build REST APIs for handling authentication, products, cart management, and orders.

Database

- PostgreSQL was used as the relational database management system.
- Supabase was used for hosting and managing the PostgreSQL database.

Authentication and Security

- JWT (JSON Web Token) was used for secure user authentication and authorization.
- bcryptjs was used for password hashing and security.

Database Concepts Used

- SQL Queries
- Primary Keys and Foreign Keys
- Joins
- Transactions (BEGIN, COMMIT)
- Triggers
- Cursors
- Stored Procedures
- Normalization